Alumni Connect Platform — Data Model

Platform schema reference

Mongoose schema reference covering every feature from the deck: profiles, alumni search, mentorship + chat + calendar, career opportunities, events, forum, success stories, AI matching, admin verification/moderation.

1. User

Single collection for all three roles (`student`, `alumni`, `admin`). Role-specific fields are simply left unset for roles that don't use them.

```
const userSchema = new Schema({
  role: { type: String, enum: ['student', 'alumni', 'admin'], required: true },
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true, trim: true, lowercase: true },
  firebaseUid: { type: String, required: true, unique: true },  // from the verified Firebase ID token —
                                                                // source of truth for auth; covers both
                                                                // email/password and Google sign-in

  // Shared profile fields
  branch: String,                 // e.g. "Computer Engineering"
  batch: Number,                  // graduation year
  skills: [String],
  linkedinUrl: String,
  githubUrl: String,
  bio: String,

  // Alumni-only
  currentCompany: String,
  currentRole: String,
  industry: String,
  areasOfExpertise: [String],
  availableForMentorshipCalls: { type: Boolean, default: false }, // controls Google Meet requests only; never chat access
  willingToRefer: { type: Boolean, default: false },  // independent of Google Meet availability
  isVerified: { type: Boolean, default: false },   // admin-verified badge

  // Student-only
  careerGoal: String,             // free-text goal used by AI matching
  resumeUrl: String,              // uploaded via RecruitSage

  createdAt: { type: Date, default: Date.now }
});

// Alumni Search
// Free-text search across the fields a student can type into the search box.
userSchema.index({
  name: 'text',
  currentCompany: 'text',
  currentRole: 'text',
  industry: 'text',
  skills: 'text',
  areasOfExpertise: 'text'
});

// General alumni browsing. Search results are not filtered by willingness to mentor or refer;
// students can filter only by profile details such as branch, batch, industry, and current role.
userSchema.index({ role: 1, isVerified: 1, branch: 1, batch: 1, industry: 1, currentRole: 1 });
```

2. MentorshipMeetingRequest

Google Meet request between a student and an alumnus. The alumnus must have `availableForMentorshipCalls: true` to receive one. This feature is separate from chat.

```
const mentorshipMeetingRequestSchema = new Schema({
  studentId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  alumniId:  { type: Schema.Types.ObjectId, ref: 'User', required: true },
  status: { type: String, enum: ['pending', 'accepted', 'rejected', 'cancelled', 'completed'], default: 'pending' },
  agenda: { type: String, required: true },        // why the student is requesting the call
  meetingScheduledAt: Date,        // set once a Google Calendar slot is booked
  googleCalendarEventId: String,   // returned by the Calendar API — used to manage/reschedule the event
  meetingLink: String,             // Google Meet join URL (conferenceData.hangoutLink from the same
                                   // Calendar API response) — saved so the frontend can show a
```

```
                                  // "Join Meeting" button without an extra API call
  createdAt: { type: Date, default: Date.now }
});

mentorshipMeetingRequestSchema.index({ studentId: 1, status: 1 });
mentorshipMeetingRequestSchema.index({ alumniId: 1, status: 1 });
// One active meeting request per pair; rejected/cancelled/completed requests remain as history and can be retried.
mentorshipMeetingRequestSchema.index(
  { studentId: 1, alumniId: 1 },
  { unique: true, partialFilterExpression: { status: { $in: ['pending', 'accepted'] } } }
);
```

3. Conversation + Message

Chat is available independently of Google Meet availability. A conversation is created when two users first message each other; no mentorship-call setting or meeting request is required.

```
const conversationSchema = new Schema({
  // Store the pair in a consistent order (smaller ObjectId first) before saving.
  participantOneId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  participantTwoId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  createdAt: { type: Date, default: Date.now }
});

// The consistent ordering makes this one private chat thread per pair of users.
conversationSchema.index({ participantOneId: 1, participantTwoId: 1 }, { unique: true });

// Required server-side rules (not just frontend checks):
// 1. Reject a conversation when participantOneId equals participantTwoId.
// 2. Put the smaller ObjectId in participantOneId before saving, so the same pair cannot create
//    a second conversation in reverse order.

const messageSchema = new Schema({
  conversationId: { type: Schema.Types.ObjectId, ref: 'Conversation', required: true },
  senderId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  text: { type: String, required: true },
  sentAt: { type: Date, default: Date.now }
});

messageSchema.index({ conversationId: 1, sentAt: 1 });

// Required server-side rule: before reading or creating a message, load its Conversation and
// reject the request unless the authenticated user's ID matches participantOneId or participantTwoId.
// This prevents anyone else from reading or posting to a private chat by using its conversationId.
```

4. Opportunity

Internships, jobs, hackathons, referrals — posted by alumni.

```
const opportunitySchema = new Schema({
  postedBy: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  type: { type: String, enum: ['internship', 'job', 'hackathon', 'referral', 'workshop'], required: true },
  title: { type: String, required: true },
  description: { type: String, required: true },
  applyLink: String,
  company: String,
  createdAt: { type: Date, default: Date.now }
});
```

4a. ReferralRequest

A student can request a referral from an alumni member who has explicitly opted in to referrals (`willingToRefer: true`) — not only someone who has posted a referral `Opportunity`. Mentorship and referrals are independent: an alumnus may offer either, both, or neither. A student browsing alumni profiles (e.g. via Alumni Search) can target someone based on where they currently work.

```
const referralRequestSchema = new Schema({
  studentId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  alumniId: { type: Schema.Types.ObjectId, ref: 'User', required: true },   // who approves/rejects
  opportunityId: { type: Schema.Types.ObjectId, ref: 'Opportunity' },       // optional — set only if
                                                                             // applying against a posted referral

  companyName: { type: String, required: true },  // usually the alumni's currentCompany, but
                                                    // stored explicitly in case it's asked at request time
  jobId: { type: String, required: true },        // job ID from the company's portal
  jobLink: { type: String, required: true },
  email: { type: String, required: true },        // contact email for this application
  resumeUrl: { type: String, required: true },
```

```
  status: { type: String, enum: ['pending', 'approved', 'rejected', 'cancelled'], default: 'pending' },
  createdAt: { type: Date, default: Date.now }
});

referralRequestSchema.index({ alumniId: 1, status: 1 });
referralRequestSchema.index({ studentId: 1, status: 1 });
// One active referral request for the same student, alumnus, and job.
referralRequestSchema.index(
  { studentId: 1, alumniId: 1, companyName: 1, jobId: 1 },
  { unique: true, partialFilterExpression: { status: { $in: ['pending', 'approved'] } } }
);
```

5. Event

Alumni meets, career talks, webinars — organized by admin or college.

```
const eventSchema = new Schema({
  organizerId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  title: { type: String, required: true },
  description: String,
  eventDate: { type: Date, required: true },
  mode: { type: String, enum: ['online', 'offline'], default: 'online' },
  link: String,        // meeting link if online
  createdAt: { type: Date, default: Date.now }
});
```

6. ForumPost + Comment

Student questions answered by alumni with real experience.

```
const forumPostSchema = new Schema({
  authorId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  question: { type: String, required: true },
  tags: [String],          // e.g. ["SDE", "Amazon", "interview"]
  createdAt: { type: Date, default: Date.now }
});

const commentSchema = new Schema({
  postId: { type: Schema.Types.ObjectId, ref: 'ForumPost', required: true },
  authorId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  text: { type: String, required: true },
  createdAt: { type: Date, default: Date.now }
});

commentSchema.index({ postId: 1, createdAt: 1 });
```

7. SuccessStory

Placement stories, internship journeys, career transitions.

```
const successStorySchema = new Schema({
  authorId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  title: { type: String, required: true },
  story: { type: String, required: true },
  category: { type: String, enum: ['placement', 'internship', 'career-transition', 'interview-experience'] },
  createdAt: { type: Date, default: Date.now }
});
```

8. Report (admin moderation)

Flags on any content type, reviewed by admin.

```
const reportSchema = new Schema({
  reportedBy: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  targetType: { type: String, enum: ['user', 'forumPost', 'comment', 'opportunity'], required: true },
  targetId: { type: Schema.Types.ObjectId, required: true },
  reason: { type: String, required: true },
  status: { type: String, enum: ['open', 'reviewed', 'dismissed'], default: 'open' },
  createdAt: { type: Date, default: Date.now }
});
```

---

9. AI Alumni Mentor Matching — no new collection needed

This is a computed feature, not stored data. On a student's query:

1. Parse `careerGoal` text (student's free-text input).
2. Score every alumni member with `availableForMentorshipCalls: true` against: career-goal keyword overlap,

`skills` overlap, `branch` match, `industry` match, `areasOfExpertise` overlap.

3. Return top N with a reason tag (`"Same Branch"`, `"Similar Journey"`, `"Industry Match"`,

`"Open to Mentor"`) derived from which score component was highest.

Start with a weighted rule-based scorer (fast, explainable, no infra). Upgrade later to embeddings (OpenAI/local model) on `careerGoal` + alumni `bio`/`areasOfExpertise`, ranked by cosine similarity — same output shape, just a smarter scoring function. No schema change needed either way; you're only ever reading from `User`.

Relationship summary

- `User` is the hub — every other collection references it via `ObjectId` (`ref: 'User'`).
- `Conversation` → `Message` (one-to-many): each private chat thread holds many messages and is independent of call availability.
- `User` (student/alumni) → `MentorshipMeetingRequest`: a student can request a Google Meet only from an alumnus with `availableForMentorshipCalls: true`.
- `User` (alumni) → `ReferralRequest` (one-to-many): a student can request a referral directly from an alumni member with `willingToRefer: true`; `opportunityId` is optional and only set when the request is tied to a posted referral opportunity. This is independent of Google Meet availability.
- `ForumPost` → `Comment` (one-to-many): kept separate to avoid unbounded array growth on posts.
- `Report` uses a polymorphic `targetType` + `targetId` pair rather than one FK per content

type — avoids adding a `reports` array to every collection just for moderation.